

En appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=PrestamosDB;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=SERVIDOR_SQL;Database=PrestamosDB;User Id=sa;Password=TuPassword123;TrustServerCertificate=True"
  }
}
```

CREAR ASP .NET CORE WEB API

```
public class Prestamo
{
    public int Id { get; set; }

    public string NombreCliente { get; set; }

    public decimal Monto { get; set; }

    public int DiasMora { get; set; }

    public int CategoriaId { get; set; }

    public DateTime FechaCreacion { get; set; }

    public Categoria Categoria { get; set; }
}
```

```

public class Categoria
{
    public int CategoriaId { get; set; }

    public string CategoriaNombre { get; set; }

    public ICollection<Prestamo> Prestamos { get; set; }
}

```

Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.5

Install-Package Microsoft.EntityFrameworkCore.Tools -Version 8.0.5

Install-Package Microsoft.AspNetCore.Authentication.JwtBearer -Version 8.0.5

```

namespace Prestamos.Api.Data
{
    public class PrestamosContext : DbContext
    {
        public PrestamosContext(DbContextOptions<PrestamosContext> options)
            : base(options)
        {
        }

        public DbSet<Prestamo> Prestamos { get; set; }

        public DbSet<Categoria> Categorias { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Prestamo>().ToTable("Prestamos");

            modelBuilder.Entity<Categoria>().ToTable("Categoria");

            modelBuilder.Entity<Categoria>()
                .HasKey(c => c.CategoriaId);

            modelBuilder.Entity<Prestamo>()
                .HasOne(p => p.Categoria)
                .WithMany(c => c.Prestamos)
                .HasForeignKey(p => p.CategoriaId);
        }
    }
}

```

Registra el DbContext en Program.cs

- Debajo de `var builder = WebApplication.CreateBuilder(args);`

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddDbContext<PrestamosContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

Scaffold-DbContext "Server=.;Database=PrestamosDB;User
Id=sa;Password=123456;TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer -
OutputDir Models -Context PrestamosContext -Tables Prestamos,Categoria -Force

Una forma aún más rápida (muy útil en pruebas técnicas)

Visual Studio puede generar este controlador automáticamente:

1. Clic derecho en Controllers → Add → New Scaffolded Item.
2. Selecciona API Controller with actions, using Entity Framework.
3. En Model class, elige `Prestamo`.
4. En Data context class, elige `PrestamosContext`.
5. Haz clic en Add.

```
alter table prestamos
add foreign key (CategoriaId)
references Categoria (CategoriaId)
```

```
ALTER TABLE Prestamos
ADD CONSTRAINT DF_Prestamos_FechaCreacion
DEFAULT GETDATE() FOR FechaCreacion;
```

```
public Categoria? Categoria { get; set; }
```

```
{  
  "nombreCliente": "Juan Pérez",  
  "monto": 2500,  
  "diasMora": 5,  
  "categoriaId": 1  
}
```

```
builder.Services.AddDbContext<PrestamosContext>(options =>  
{  
    options.UseSqlServer(  
        builder.Configuration.GetConnectionString("DefaultConnection"));  
});  
  
builder.Services.AddCors(options =>  
{  
    options.AddPolicy("AngularPolicy", policy =>  
    {  
        policy.WithOrigins("http://localhost:4200")  
            .AllowAnyHeader()  
            .AllowAnyMethod();  
    });  
});
```

```
// Habilita CORS  
app.UseCors("AngularPolicy");  
  
app.UseHttpsRedirection();  
  
app.UseAuthorization();  
  
app.MapControllers();  
  
app.Run();
```

¿Por qué se usan GET, POST, PUT y DELETE?

Estos métodos forman parte del estándar HTTP y cada uno tiene un propósito específico. Las API REST siguen ese estándar para que cualquier cliente (Angular, React, Flutter, Postman, etc.) sepa qué operación está realizando.

Método HTTP	Operación	Equivalente en SQL
GET	Consultar información	SELECT
POST	Crear un nuevo recurso	INSERT
PUT	Actualizar un recurso existente	UPDATE
DELETE	Eliminar un recurso	DELETE

"Una API REST es una interfaz que permite que diferentes aplicaciones se comuniquen entre sí utilizando el protocolo HTTP. Expone recursos, como clientes, préstamos o productos, a través de URLs y utiliza los métodos HTTP como GET, POST, PUT y DELETE para consultar, crear, actualizar o eliminar información. Generalmente intercambia datos en formato JSON, lo que facilita que aplicaciones web, móviles o de escritorio puedan consumirla sin importar el lenguaje en que estén desarrolladas."

Si pregunta: "¿Qué ventajas tiene?"

Puedes mencionar estas:

- Independencia entre frontend y backend.
- Puede ser consumida por cualquier tecnología que entienda HTTP.
- Generalmente utiliza JSON, que es un formato ligero y fácil de interpretar.
- Facilita el mantenimiento y la escalabilidad.
- Permite implementar seguridad mediante JWT, OAuth, API Keys, etc.
- Es reutilizable: una sola API puede atender varios clientes.

CRUD CON ADO.NET

```

namespace PrestamosSolution.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 referencia
    public class PrestamoController : ControllerBase
    {

        private readonly IConfiguration _configuration;
        private readonly string _cadena;

        0 referencias
        public PrestamoController(IConfiguration configuration)
        {
            _configuration = configuration;
            _cadena = _configuration.GetConnectionString("DefaultConnection");
        }

        [HttpGet("{id}")]
    }
}

```

```

[HttpGet("{id}")]
0 referencias
public IActionResult Get(int id)
{
    Prestamo? prestamo = null;

    SqlConnection cn = new SqlConnection(_cadena);
    string sql = @"SELECT Id,NombreCliente,Monto, DiasMora,CategoriaId,FechaCreacion FROM Prestamos WHERE Id = @Id";

    SqlCommand cm = new SqlCommand(sql, cn);

    cm.Parameters.AddWithValue("id", id);

    cn.Open();
    SqlDataReader dr = cm.ExecuteReader();
    if (dr.Read())
    {
        prestamo = new Prestamo
        {
            Id = Convert.ToInt32(dr["Id"]),
            NombreCliente = dr["NombreCliente"].ToString(),
            Monto = Convert.ToDecimal(dr["Monto"]),
            DiasMora = Convert.ToInt32(dr["DiasMora"]),
            CategoriaId = Convert.ToInt32(dr["CategoriaId"]),
        };
    }

    if (prestamo == null)
        return NotFound();

    return Ok(prestamo);
}

```

CON PROCEDIMIENTO ALMACENADO

```
[HttpGet("{id}")]
public IActionResult Get(int id)
{
    Prestamo? prestamo = null;

    using (SqlConnection cn = new SqlConnection(_cadena))
    {

        SqlCommand cmd = new SqlCommand(
            "sp_ObtenerPrestamoPorId",
            cn
        );

        // Indicamos que es un procedimiento almacenado
        cmd.CommandType = CommandType.StoredProcedure;

        // Enviamos el parámetro
        cmd.Parameters.AddWithValue("@Id", id);

        cn.Open();

        SqlDataReader dr = cmd.ExecuteReader();
```

```
[HttpPost]
public IActionResult Post(Prestamo prestamo)
{
    using(SqlConnection cn = new SqlConnection(_cadena))
    {
        SqlCommand cmd = new SqlCommand(
            "sp_InsertarPrestamo",
            cn
        );

        // Indicamos que es procedimiento almacenado
        cmd.CommandType = CommandType.StoredProcedure;

        // Enviamos parámetros
        cmd.Parameters.AddWithValue(
            "@NombreCliente",
            prestamo.NombreCliente
        );

        cmd.Parameters.AddWithValue(
            "@Monto",
            prestamo.Monto
        );

        cmd.Parameters.AddWithValue(
            "@DiasMora",
            prestamo.DiasMora
        );
    }
}
```




```
cn.Open();

int filas = cmd.ExecuteNonQuery();

if(filas > 0)
{
    return Ok(new
    {
        mensaje = "Préstamo registrado correctamente"
    });
}

return BadRequest(
    "No se pudo insertar el registro"
);
}
}
```

```

[HttpGet]
0 referencias
public IActionResult Get()
{
    List<Prestamo> lista = new List<Prestamo>();

    SqlConnection cn = new SqlConnection(_cadena);

    try
    {
        string sql = @"SELECT
                        Id,
                        NombreCliente,
                        Monto,
                        DiasMora,
                        CategoriaId,
                        FechaCreacion
                        FROM Prestamos";

        SqlCommand cmd = new SqlCommand(sql, cn);

        cn.Open();

        SqlDataReader dr = cmd.ExecuteReader();

        while (dr.Read())
        {
            lista.Add(new Prestamo
            {
                Id = Convert.ToInt32(dr["Id"]),
                NombreCliente = dr["NombreCliente"].ToString(),
                Monto = Convert.ToDecimal(dr["Monto"]),
                DiasMora = Convert.ToInt32(dr["DiasMora"]),
                CategoriaId = Convert.ToInt32(dr["CategoriaId"]),
            });
        }
    }
}

```

```

        dr.Close();
        cmd.Dispose();
        cn.Close();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
    finally
    {
        cn.Dispose();
    }

    return Ok(lista);
}

```

```

[HttpPost]
0 referencias
public IActionResult Post(Prestamo prestamo)
{
    using (SqlConnection cn = new SqlConnection(_cadena))
    {
        string sql = @"INSERT INTO Prestamos
        (
            NombreCliente,
            Monto,
            DiasMora,
            CategoriaId,
            FechaCreacion
        )
        VALUES
        (
            @NombreCliente,
            @Monto,
            @DiasMora,
            @CategoriaId,
            GETDATE()
        )";

        SqlCommand cmd = new SqlCommand(sql, cn);

        cmd.Parameters.AddWithValue("@NombreCliente", prestamo.NombreCliente);
        cmd.Parameters.AddWithValue("@Monto", prestamo.Monto);
        cmd.Parameters.AddWithValue("@DiasMora", prestamo.DiasMora);
        cmd.Parameters.AddWithValue("@CategoriaId", prestamo.CategoriaId);

        cn.Open();

        cmd.ExecuteNonQuery();
    }

    return Ok("Registro agregado correctamente");
}

```

```

return Ok(new
{
    mensaje = "Registro agregado correctamente"
});

```

```

[HttpPut("{id}")]
0 referencias
public IActionResult Put(int id, Prestamo prestamo)
{
    using (SqlConnection cn = new SqlConnection(_cadena))
    {
        string sql = @"UPDATE Prestamos
                        SET
                            NombreCliente=@NombreCliente,
                            Monto=@Monto,
                            DiasMora=@DiasMora,
                            CategoriaId=@CategoriaId
                        WHERE Id=@Id";

        SqlCommand cmd = new SqlCommand(sql, cn);

        cmd.Parameters.AddWithValue("@Id", id);
        cmd.Parameters.AddWithValue("@NombreCliente", prestamo.NombreCliente);
        cmd.Parameters.AddWithValue("@Monto", prestamo.Monto);
        cmd.Parameters.AddWithValue("@DiasMora", prestamo.DiasMora);
        cmd.Parameters.AddWithValue("@CategoriaId", prestamo.CategoriaId);

        cn.Open();

        int filas = cmd.ExecuteNonQuery();

        if (filas == 0)
            return NotFound();

        return Ok("Registro actualizado");
    }
}

```

```

[HttpDelete("{id}")]
0 referencias
public IActionResult Delete(int id)
{
    using (SqlConnection cn = new SqlConnection(_cadena))
    {
        string sql = "DELETE FROM Prestamos WHERE Id=@Id";

        SqlCommand cmd = new SqlCommand(sql, cn);

        cmd.Parameters.AddWithValue("@Id", id);

        cn.Open();

        int filas = cmd.ExecuteNonQuery();

        if (filas == 0)
            return NotFound();

        return Ok("Registro eliminado");
    }
}

```

EJEMPLO PARA CONSUMIR DESDE SITIO WEB MVC .NET

```
<configuration>
  <configSections>

    <appSettings>
<add key="Conexion" value="Data Source=10.0.0.22;Initial Catalog=cac;User
Id=accesoinfo2;Password=67Mustang"/>
    </appSettings>
```

Librería recomendadas

html

```
<!-- 1. Estilos CSS (Para el diseño visual de la tabla) -->
<link rel="stylesheet" href="https://datatables.net">

<!-- 2. jQuery (Esencial para que funcione el selector '$') -->
<script src="https://jquery.com"></script>

<!-- 3. Script Core de DataTables -->
<script src="https://datatables.net"></script>
```

```
function ObtenerDatos() {
    $.post("../PuntoAtencion/MostrarPuntosAtencion", {})
    .done(function (data) {
        datos_global = _.groupBy(data, "id");

        Grid_Datos(data);
    }).
    fail(function (ex) {
        console.warn(ex);
    }).always(function () {
    });
}
```

```
function Grid_Datos(d) {
    var tds = "";
    for (var i = 0, len = d.length; i < len; i++) {

        tds = tds + "<tr><td>" + d[i].id + "</td>" +
            "<td>" + d[i].sucursal + "</td>" +
            "<td>" + d[i].direccion + "</td>" +
            "<td>" + d[i].contacto + "</td>" +
            "<td>" + d[i].correo + "</td>" +
            "<td>" + d[i].telefono + "</td>" +
            "</tr>";

    }

    var thead = "<thead><tr>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='10%'>Codigo Sucursal</th>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='15%'>Sucursal</th>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='15%'>Dirección</th>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='15%'>Contacto</th>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='15%'>Correo</th>" +
        "<th class='p-3 mb-2 bg-primary text-white' width='15%'>Telefono</th>" +
        "</tr></thead>";

    $("#tblData").html("<table id='tblData' class='table table-striped table-bordered' cellspacing='0' width='100%'>" + thead + "<tbody>" + tds + "</tbody>" + "</table>");
}
```

```
$('#tblData').DataTable({
    "language": {
        "sProcessing": "Procesando...",
        "sLengthMenu": "Mostrar _MENU_ registros",
        "sZeroRecords": "No se encontraron resultados",
        "sEmptyTable": "Ningún dato disponible en esta tabla",
        "sInfo": "Mostrando registros del _START_ al _END_ de un total de _TOTAL_ registros",
        "sInfoEmpty": "Mostrando registros del 0 al 0 de un total de 0 registros",
        "sInfoFiltered": "(filtrado de un total de _MAX_ registros)",
        "sInfoPostFix": "",
        "sSearch": "Buscar:",
        "sUrl": "",
        "sInfoThousands": ",",
        "sLoadingRecords": "Cargando...",
        "oPaginate": {
            "sFirst": "Primero",
            "sLast": "Último",
            "sNext": "Siguiente",
            "sPrevious": "Anterior"
        },
        "oAria": {
            "sSortAscending": ": Activar para ordenar la columna de manera ascendente",
            "sSortDescending": ": Activar para ordenar la columna de manera descendente"
        }
    }
});
```

```
$.post("../PuntoAtencion/ModificarPuntoAtencion", { CodSucursal:id_global,nombre: punto_atencion, direccion: direccion, contacto: contacto, correo: correo, telefono: telefono })
.done(function (data) {
    mensaje = parseInt(data[0].Resultado);
    if (mensaje === 1) {
        limpiar();
        ObtenerDatos();
        $(".modal-content .close").click();
        alertify.success("Punto de Atención modificado correctamente");
    }
    else
    {
        alertify.error("Error al modificar centro de atención");
    }
});
}).fail(function (ex) {
    limpiar();
    alertify.error("Error Favor revisar los parametros enviados");
}).always(function () {
});
});
```

```
<div id="tbl_Data" class="table-responsive"></div>
```

```
public ActionResult MostrarPuntosAtencion()
{
    List<Sucursal> lista = new List<Sucursal>();
    DataTable dtPuntosAtencion = new DataTable();
    SqlConnection con = new SqlConnection(conexion);
    string consulta = "";
    consulta = "select * from grlSucursales";

    SqlDataAdapter adpSucursales = new SqlDataAdapter("", con);
    adpSucursales.SelectCommand.CommandText=consulta;

    adpSucursales.Fill(dtPuntosAtencion);

    for (int i = 0; i < dtPuntosAtencion.Rows.Count;i++ )
    {
        lista.Add(new Sucursal()
        {
            id = Convert.ToInt32(dtPuntosAtencion.Rows[i]["CodSucursal"].ToString()),
            sucursal = dtPuntosAtencion.Rows[i]["Nombre"].ToString(),
            direccion = dtPuntosAtencion.Rows[i]["Direccion"].ToString(),
            contacto = dtPuntosAtencion.Rows[i]["contacto"].ToString(),
            correo = dtPuntosAtencion.Rows[i]["correo"].ToString(),
            telefono = dtPuntosAtencion.Rows[i]["telefono"].ToString()
        });
    }

    return Json(lista, JsonRequestBehavior.AllowGet);
}
```

```
string conexion = ConfigurationManager.AppSettings["Conexion"].ToString();  
//
```

"Desde el frontend realizo una petición HTTP utilizando una librería como jQuery AJAX, Angular HttpClient o Fetch API. Envío la solicitud al endpoint de la API REST, recibo la respuesta JSON y luego transformo esos datos para mostrarlos en la interfaz."

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

```
function listarPrestamos() {  
  
    $.ajax({  
  
        url: "https://localhost:7260/api/prestamos",  
        type: "GET",  
        dataType: "json",  
  
        success: function (data) {  
  
            let filas = "";  
  
            $.each(data, function (index, prestamo) {  
  
                filas += `  
                    <tr>  
                        <td>${prestamo.id}</td>  
                        <td>${prestamo.nombreCliente}</td>  
                        <td>${prestamo.monto}</td>  
                        <td>${prestamo.diasMora}</td>  
                        <td>${prestamo.categoriaId}</td>  
                        <td>${prestamo.fechaCreacion}</td>  
                    </tr>  
                `;  
  
            });  
  
            $("#tablaPrestamos").html(filas);  
        }  
    });  
}
```




```

    },

    error: function (error) {

        console.log("Error al consultar API");
        console.log(error);

    }

});

}

```

Ejecutarla cuando cargue la página

JavaScript

```

$(document).ready(function(){

    listarPrestamos();

});

```



```

<table class="table table-bordered">
  <thead>
    <tr>
      <th>ID</th>
      <th>Cliente</th>
      <th>Monto</th>
      <th>Días Mora</th>
      <th>Categoria</th>
      <th>Fecha</th>
    </tr>
  </thead>

  <tbody id="tablaPrestamos">
  </tbody>
</table>

```

```

<div class="container">

  <h3>Nuevo Préstamo</h3>

  <div class="mb-3">
    <label>Nombre Cliente</label>
    <input type="text" id="nombreCliente" class="form-control">
  </div>

  <div class="mb-3">
    <label>Monto</label>
    <input type="number" id="monto" class="form-control">
  </div>

  <div class="mb-3">
    <label>Días Mora</label>
    <input type="number" id="diasMora" class="form-control">
  </div>

```

```

function guardarPrestamo() {

  let prestamo = {

    nombreCliente: $("#nombreCliente").val(),

    monto: parseFloat($("#monto").val()),

    diasMora: parseInt($("#diasMora").val()),

    categoriaId: parseInt($("#categoriaId").val())

  };

  $.ajax({

    url: "https://localhost:7260/api/prestamos",

    type: "POST",

    contentType: "application/json; charset=utf-8",

    data: JSON.stringify(prestamo),

    success: function (respuesta) {

      alert("Préstamo guardado correctamente")

      listarPrestamos(); // refrescar tabla
    }
  });
}

```



```

        limpiarFormulario();

    },

    error: function (error) {

        console.log(error);

        alert("Error al guardar");

    }

});

}

```

```

WITH VentasCompania AS (
    -- Sucursal Norte
    SELECT 'Norte' AS Sucursal, 'Carlos' AS Empleado, 5000 AS TotalVenta UNION ALL
    SELECT 'Norte' AS Sucursal, 'Ana' AS Empleado, 8500 AS TotalVenta UNION ALL
    SELECT 'Norte' AS Sucursal, 'Luis' AS Empleado, 3000 AS TotalVenta UNION ALL
    -- Sucursal Sur
    SELECT 'Sur' AS Sucursal, 'Sofia' AS Empleado, 9000 AS TotalVenta UNION ALL
    SELECT 'Sur' AS Sucursal, 'Maria' AS Empleado, 4500 AS TotalVenta UNION ALL
    -- Sucursal Este
    SELECT 'Este' AS Sucursal, 'Pedro' AS Empleado, 7000 AS TotalVenta UNION ALL
    SELECT 'Este' AS Sucursal, 'Lucia' AS Empleado, 9500 AS TotalVenta
)
SELECT
    ROW_NUMBER() OVER (
        PARTITION BY Sucursal
        ORDER BY TotalVenta DESC
    ) AS RankingSucursal,
    Sucursal,
    Empleado,
    TotalVenta
FROM VentasCompania;

```

```

;WITH VentasCompania AS (
  -- Sucursal Norte
  SELECT 'Norte' AS Sucursal, 'Carlos' AS Empleado, 5000 AS TotalVenta
),
resumen as (
SELECT
  ROW_NUMBER() OVER (
    PARTITION BY Sucursal
    ORDER BY TotalVenta DESC
  ) AS RankingSucursal,
  Sucursal,
  Empleado,
  TotalVenta
FROM VentasCompania)

select * from resumen where RankingSucursal=1;

```

ANGULAR

EN APP.HTML

```
<router-outlet></router-outlet>
```

ng new prestamos-app

cd prestamos-app

npm install bootstrap

Luego en **angular.json**

```

"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/styles.css"
]

```

ng generate service services/préstamo.service

ng generate component pages/prestamos

ng generate component pages/prestamo-form

ng generate interface models/prestamo

El servicio necesario para llamar la api

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Prestamo } from '../models/prestamo';
@Injectable({
  providedIn: 'root',
})
export class PrestamoService {

  private apiUrl = "https://localhost:7260/api/Prestamo";
  constructor(private http: HttpClient) {}

  getPrestamos(): Observable<Prestamo[]> {
    return this.http.get<Prestamo[]>(this.apiUrl);
  }

  obtenerPrestamo(id:number): Observable<Prestamo>{
    return this.http.get<Prestamo>(`${this.apiUrl}/${id}`);
  }

  crearPrestamo(data:Prestamo){
    return this.http.post(this.apiUrl,data);
  }

  actualizarPrestamo(id:number,data:Prestamo){
    return this.http.put(`${this.apiUrl}/${id}`,data);
  }

  eliminarPrestamo(id:number){
    return this.http.delete(`${this.apiUrl}/${id}`);
  }
}
```

Ahora Angular usa **providers** en app.config.ts o main.ts.

```
import { provideHttpClient } from '@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient()
  ]
});
```

En app.module.ts

```

</> TypeScript

import { HttpClientModule } from '@angular/common/http';

imports: [
  HttpClientModule
]

```

Si tu proyecto usa **AppModule**, debe ir en:

```

</> TypeScript

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    HttpClientModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

En ese caso **no existe** `app.module.ts`.

Entonces se usa:

`main.ts`

```

</> TypeScript

import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient()
  ]
});

```

app.config.ts

```
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes), provideClientHydration(withEventReplay()),
    provideHttpClient()
  ]
};
```

En app.routes.ts

```
import { Routes } from '@angular/router';
import { Prestamos } from '../pages/prestamos/prestamos';
import { PrestamoForm } from '../pages/prestamo-form/prestamo-form';

export const routes: Routes = [
  { path: '', component: Prestamos },
  { path: 'prestamos', component: Prestamos },
  { path: 'prestamos/nuevo', component: PrestamoForm },
  { path: 'prestamos/editar/:id', component: PrestamoForm },
];
```

App.ts

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('inter');
}
```

Prestamos.html


```

<div class="d-flex justify-content-center">
  <button class="btn btn-primary mb-3" routerLink="/prestamos/nuevo">
    | Nuevo préstamo
  </button>
</div>
<br>
<div class="container mt-4">
<h3>Lista de Préstamos</h3>
<table class="table table-bordered table-striped">
<thead>
<tr>
<th>Cliente</th>
<th>Monto</th>
<th>Días Mora</th>
<th>Acciones</th>
</tr>
</thead>
<tbody>
<tr *ngFor="let p of prestamos">
<td>{{p.nombreCliente}}</td>
<td>{{p.monto}}</td>
<td>{{p.diasMora}}</td>
<td>
<button class="btn btn-warning btn-sm"
(click)="editar(p.id)">
  Editar
</button>
<button class="btn btn-danger btn-sm"
(click)="eliminar(p.id)">
  Eliminar
</button></td></tr></tbody></table></div>

```

Prestamos.ts

```
import { CommonModule } from '@angular/common';
import { Component, OnInit, ChangeDetectorRef } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { PrestamoService } from '../../services/prestamo.service';
@Component({
  selector: 'app-prestamos',
  imports: [CommonModule, RouterLink],
  standalone: true,
  templateUrl: './prestamos.html',
  styleUrls: ['./prestamos.css'],
})
export class Prestamos implements OnInit {
  prestamos: any[] = [];
  constructor(
    private router: Router,
    private servicio: PrestamoService,
    private cd: ChangeDetectorRef
  ) {}

  ngOnInit(): void {
    this.cargar();
  }
}
```

```

cargar() {
  this.servicio.getPrestamos().subscribe({
    next: (data) => {
      this.prestamos = data;
      this.cd.detectChanges(); // ◆ fuerza actualización de la vista
    },
    error: (err) => {
      console.error("Error API:", err);
    }
  });
}

editar(id: number) {
  this.router.navigate(['/prestamos/editar', id]);
}

eliminar(id: number) {
  if (confirm("Desea eliminar?")) {
    this.servicio.eliminarPrestamo(id)
      .subscribe(() => this.cargar());
  }
}
}

```

Prestamo-form.html

```
<div class="container mt-4">
<h3>{{esEdicion ? 'Editar Prestamo' : 'Nuevo Prestamo'}}</h3>
<form (ngSubmit)="guardar()" #form="ngForm">
<div class="mb-3">
<label>Cliente</label>

<input type="text"
class="form-control"
name="nombreCliente"
[(ngModel)]="prestamo.nombreCliente"
required>

</div>

<div class="mb-3">
<label>Monto</label>

<input type="number"
class="form-control"
name="monto"
[(ngModel)]="prestamo.monto">

</div>
```

```

<div class="mb-3">

<label>Dias Mora</label>

<input type="number"
class="form-control"
name="diasMora"
[(ngModel)]="prestamo.diasMora">

</div>

<div class="mb-3">

<label>Categoria</label>

<select class="form-control"
name="categoriaId"
[(ngModel)]="prestamo.categoriaId">

  <option value="">Seleccione categoría</option>

  <option *ngFor="let c of categorias" [value]="c.id">
    {{c.nombre}}
  </option>
</select>

</div>

```

```

<button class="btn btn-success">
  Guardar
</button>

</form>

</div>

```

Prestamo-form.ts

```
import { CommonModule } from '@angular/common';
import { ChangeDetectorRef, Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { ActivatedRoute } from '@angular/router';
import { PrestamoService } from '../../services/prestamo.service';

@Component({
  selector: 'app-prestamo-form',
  imports: [FormsModule, CommonModule],
  standalone: true,
  templateUrl: './prestamo-form.html',
  styleUrls: ['./prestamo-form.css'],
})
export class PrestamoForm {

  esEdicion=false;
  idPrestamo:number=0;

  categorias:any[] = [
    { id: 1, nombre: 'A' },
    { id: 2, nombre: 'B' },
    { id: 3, nombre: 'C' },
    { id: 4, nombre: 'D' },
    { id: 5, nombre: 'E' }
  ];
}
```

```

export class PrestamoForm {
  prestamo: any = {
    monto: 0,
    diasMora: 0,
    categoriaId: ''
  };
}

constructor(
  private router: Router,
  private route: ActivatedRoute,
  private service: PrestamoService,
  private cd: ChangeDetectorRef
){}

ngOnInit(){

  this.idPrestamo = Number(this.route.snapshot.paramMap.get('id'));

  if(this.idPrestamo){
    this.cargarPrestamo();
    this.esEdicion=true;
  }

}

```

```

cargarPrestamo(){

  this.service.obtenerPrestamo(this.idPrestamo)
    .subscribe(data=>{
      console.log("Datos del préstamo:", data);
      this.prestamo = data;
      this.cd.detectChanges();
    });

}

```

```
guardar(){

  this.prestamo.categoriaId = Number(this.prestamo.categoriaId);

  if(this.idPrestamo){

    this.prestamo.id = this.idPrestamo; // ➡ agregar esto

    this.service.actualizarPrestamo(this.idPrestamo,this.prestamo)
    .subscribe({
      next: ()=>{
        alert("Prestamo actualizado");
        this.router.navigate(['/prestamos']);
      },
      error:(err)=>{
        console.Log(err);
      }
    });
  } else {

    this.service.crearPrestamo(this.prestamo)
    .subscribe(()=>{
      alert("Prestamo guardado");
      this.router.navigate(['/prestamos']);
    });
  }
}
```